

R Syntax Reference: readr, dplyr, lubridate & base R

A syntax reference, not a solution key — every example below uses generic column/table names (`df`, `id`, `category`, `value`, `date_col`). None of it maps directly onto the assignment's specific logic. Swap in the real names and think through the logic yourself.

Charts are not covered here. For `qic()` syntax, chart types ("`run`", "`p`", "`i`", etc.), and how to read `sigma.signal` / `runs.signal` from the result, see the `qicharts2` package documentation: run `?qic` in the console, or browse <https://anhoej.github.io/qicharts2/>.

```
library(readr)
library(dplyr)
library(lubridate)
```

1. Reading and inspecting data

Task	Syntax
Read a CSV	<code>df <- read_csv("path/to/file.csv")</code>
Peek at structure + types	<code>glimpse(df)</code>
First/last rows	<code>head(df)</code> / <code>tail(df)</code>
Column names	<code>names(df)</code>
Count rows	<code>nrow(df)</code>
Scroll through rows in a viewer	<code>View(df)</code>
Print more rows in console	<code>print(df, n = 30)</code>
Number of distinct values	<code>n_distinct(df\$col)</code>

Reading column types. `read_csv()` guesses each column's type from a sample of rows. Check what it guessed — a column you expect to be a date or number that came in as `<chr>` is a signal something in that column doesn't match a single consistent format.

2. Selecting, filtering, creating columns

```
df |> select(col_a, col_b)           # keep specific columns
df |> select(-col_a)                 # drop a column
df |> rename(new_name = old_name)    # rename a column
df |> filter(category == "x")        # keep rows matching a condition
df |> filter(value > 10, value < 20) # multiple conditions = AND
df |> filter(category %in% c("x","y")) # match any of several values
df |> mutate(new_col = value * 2)    # add/overwrite a column
```

```
df |> distinct(id)           # unique values of one column
df |> distinct(id, date_col, value) # unique combinations of columns
```

filter() vs **distinct()**. **filter()** picks rows meeting a logical condition. **distinct()** collapses rows that are identical across the columns you name — useful when a dataset has exact duplicate entries.

Comparison/logical operators: **==**, **!=**, **>**, **>=**, **<**, **<=**, **%in%**, **&** (and), **|** (or), **!** (not), **is.na(x)** / **!is.na(x)**.

3. Grouped summaries and counting

```
df |>
  group_by(category) |>
  summarise(
    n      = n(),           # count of rows
    total  = sum(value, na.rm = TRUE),
    avg    = mean(value, na.rm = TRUE),
    .groups = "drop"
  )

df |> count(category)      # shortcut for group_by + n()
df |> count(category, date_col) # count per combination of columns
```

n() inside **summarise()** counts rows in the current group — no argument needed. **count()** is a convenient shortcut when all you want is a row count per group.

4. "Keep one row per group" logic

Sometimes you don't want a *summary* — you want to keep one specific **full row** per group (e.g., the most recent, the largest, the first).

```
df |>
  group_by(id) |>
  slice_max(date_col, n = 1, with_ties = FALSE) |> # row with max date, per id
  ungroup()

df |>
  group_by(id) |>
  slice_min(value, n = 1, with_ties = FALSE) |>    # row with min value, per id
  ungroup()

df |> arrange(id, date_col)      # sort ascending
df |> arrange(desc(date_col))    # sort descending

df |> mutate(t = row_number())   # 1, 2, 3, ... in current row order
```

`with_ties = FALSE` guarantees exactly one row per group even if there's a tie — otherwise ties are all kept, which can silently inflate your row count. `row_number()` is handy for building a simple time index (e.g., month 1, 2, 3, ...) once your rows are correctly sorted.

5. Joining two tables

```
left_join(df_a, df_b, by = "id")
left_join(df_a, df_b, by = c("id_a" = "id_b")) # different column names on each
side
inner_join(df_a, df_b, by = "id")
```

- `left_join(a, b)` keeps every row of `a`, and attaches matching columns from `b` where a match exists. Rows in `a` with **no** match in `b` are kept, with `NA` filled in for `b`'s columns — this is often exactly what you want when "no match" is itself meaningful information, not an error.
- `inner_join(a, b)` keeps only rows that matched in *both* tables — rows in `a` without a match disappear entirely.
- **Always check row counts and NAs before and after a join.** `nrow()` before/after, and `sum(is.na(df$new_col))` afterward. An unexpected jump in row count usually means the "key" column wasn't actually unique in one of the tables; unexpected `NAs` after a `left_join` usually mean the key values don't match exactly between the two tables (see the string-cleaning section below).

6. Working with dates (`lubridate`)

```
as.Date("2024-01-15") # ISO format parses by default
as.Date("01/15/2024", format = "%m/%d/%Y") # tell it the format explicitly
as.Date("15-Jan-2024", format = "%d-%b-%Y")

year(date_col) # extract year
month(date_col) # extract month
floor_date(date_col, "month") # snap any date to the 1st of its
month
difftime(date_a, date_b, units = "days") # difference between two dates, in
days
```

Parsing a column with more than one date format. `as.Date(x, format = ...)` returns `NA` for any string that doesn't match the format you gave it — it won't error, it just quietly fails on the mismatches. `coalesce()` (from `dplyr`) fills `NAs` in one vector with values from another, position by position:

```
attempt_1 <- as.Date(x, format = "%Y-%m-%d") # NA wherever this format doesn't
match
attempt_2 <- as.Date(x, format = "%m/%d/%Y") # NA wherever THIS format doesn't
match
```

```
coalesce(attempt_1, attempt_2)      # takes attempt_1 unless it's NA,
then attempt_2
```

floor_date() as a grouping key. When you need a monthly count from data with many different dates in the same month, `floor_date(date_col, "month")` turns every date into the first of its month — a clean column to `group_by()` on.

Age from a birth date, as of some reference date:

```
as.numeric(difftime(reference_date, birth_date, units = "days")) / 365.25
```

7. Cleaning messy values (base R + dplyr)

```
as.numeric(x)      # converts text to number; non-numeric text becomes NA
(with a warning)
is.na(x)           # TRUE where a value is missing
sum(is.na(df$col)) # how many missing values in a column
coalesce(x, y)     # first non-NA value, position by position
if_else(condition, true_value, false_value) # vectorized if/else, type-safe
```

A warning is not an error. `as.numeric("pending")` will run successfully and produce `NA`, with a console message telling you it happened — read those messages, don't just clear the console.

A value that converts fine can still be wrong. `as.numeric("78")` succeeds and gives you the number `78` — nothing about that operation flags it as implausible for, say, a lab value that's normally under 15. Catching this requires a **separate plausibility check**, not more type conversion:

```
df |> filter(value >= 4, value <= 15) # keep only physiologically plausible
values
```

8. Cleaning text/ID values (base R)

```
toupper(x)      # convert to upper case
tolower(x)      # convert to lower case
trimws(x)       # remove leading/trailing whitespace
paste0(x, y)    # concatenate strings with no separator
sprintf("%s-%03d", x, 7) # formatted string building, e.g. "ID-007"
grepl("pattern", x) # TRUE/FALSE: does x contain this pattern?
```

Standardizing a join key. If a join is silently dropping rows, inconsistent capitalization or stray whitespace in the key column on one or both sides is a common culprit. A safe habit is to standardize both sides before

joining, even if only one looks messy:

```
df_a <- df_a |> mutate(key_std = toupper(trimws(key)))
df_b <- df_b |> mutate(key_std = toupper(trimws(key)))
left_join(df_a, df_b, by = "key_std")
```

9. A simple linear model (base R)

```
model <- lm(y ~ x1 + x2 + x3, data = df)
summary(model)           # coefficients, standard errors, p-values, R-squared
coef(summary(model))     # just the coefficient table, as a matrix
```

Reading `summary(model)` output: each row under **Coefficients** is one term in the model. **Estimate** is the fitted coefficient; **Std. Error** is its uncertainty; **Pr(>|t|)** is the p-value for testing whether that coefficient is different from zero (conventionally, **< 0.05** is treated as statistically significant — R marks these with *).

Building indicator (0/1) and interaction variables for a segmented regression:

```
df <- df |>
  mutate(
    t      = row_number(),           # simple time counter
    post   = as.integer(date_col >= as.Date("2025-03-01")), # 0/1 indicator
    t_post = post * (t - min(t[post == 1]) + 1)           # interaction, reset
    to count from the cutoff
  )
```

`as.integer(TRUE/FALSE)` converts a logical condition directly into a 0/1 numeric column, which is what a regression indicator needs.

Quick lookup

I want to...	Function
Read a CSV	<code>read_csv()</code>
See column types / preview	<code>glimpse()</code>
Keep rows matching a condition	<code>filter()</code>
Keep/drop/rename columns	<code>select()</code> / <code>select(-x)</code> / <code>rename()</code>
Add/change a column	<code>mutate()</code>
Remove exact duplicate rows	<code>distinct()</code>
Summarize per group	<code>group_by()</code> + <code>summarise()</code>

I want to...	Function
Count rows per group	<code>count()</code>
Keep the whole "top" row per group	<code>group_by()</code> + <code>slice_max()</code>
Sort rows	<code>arrange()</code>
Build a simple row counter	<code>row_number()</code>
Combine two tables by a shared column	<code>left_join()</code> / <code>inner_join()</code>
Parse a date string	<code>as.Date(x, format = ...)</code>
Snap a date to the start of its month	<code>floor_date(x, "month")</code>
Fill NAs from a backup vector	<code>coalesce()</code>
Extract year/month from a date	<code>year()</code> / <code>month()</code>
Convert text to number safely	<code>as.numeric()</code>
Count NAs in a column	<code>sum(is.na(x))</code>
Vectorized if/else	<code>if_else()</code>
Standardize a text/ID column	<code>toupper()</code> , <code>tolower()</code> , <code>trimws()</code>
Build a 0/1 indicator from a condition	<code>as.integer(condition)</code>
Fit a linear regression	<code>lm(y ~ x, data = df)</code>
See regression results	<code>summary(model)</code>